L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #4 Due: 2025-04-22

Learning Objectives

In this assignment you will demonstrate:

- ✅ A familiarity with the basics of the four major computation theories
- ✅ A decent understanding of formal language theory and its importance within computer science
- ✅ Knowledge of the foundations of Automata Theory and Computation Theory, including the Chomsky Hierarchy, the difference between decidability and acceptability.
- ✅ The ability to craft Turing Machines for various programming and decision problems
- ✅ Working knowledge of register machines both as a model of computation and as a target of compilation
- ✅ An understanding of intermediate languages and virtual machines
- ✅ The ability to write the back end of a compiler, targeting a virtual machine or a high level language

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Do any of the assigned readings and watchings from the first three assignments that you did not get to.
- ✅ Work through the course notes on How to Write a Compiler, Theories of Computer Science, Language Theory, Automata Theory, Turing Machines, Register Machines, and Computability Theory.

- ✅ Read Ch. 6–8 in [Bernhardt]
- ✅ Read Ch. 4 of [Sipser], and skim Ch. 5–6
- ✅ Read Ch. 7 of [Mogenson]
- ✅ To the best of your ability, as time allows, skim Turing's most famous paper
- ✅ Skim the Wikipedia articles on Turing Machines and Undecidability
- ✅ Skim the Wikipedia articles on Transpilers, Intermediate Language, Virtual Machine, LLVM, Java Virtual Machine, C--, WebAssembly, Single Static Assignment

Submission Instructions

As usual, turn in a single submission for your entire group via BrightSpace. Remember to not simply partition the work among team members, as this reduces your learning.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project

Exercises

1. In your own words, write one sentence for each of the four major theories of computation, conveying its central question and its areas of concern. Write as if your job depended on clarity, accuracy, and solid English writing skills. If you look to an AI assistant for help, do not just copy what the bot says; but feel free to try so see if it can even get close on this one. The scope of the four theories are kind of fuzzy, so stick with the definitions you've heard in class rather than the bot's training set.

2. Given $L_1 = \{0, 011, 10\}$ and $L_2 = \{10, 1\}$. What are:
   a. $L_1 \cup L_2$
   b. $L_1 \cap L_2$
   c. $L_1 L_2$

d. $L_2{}^*$

3. Give grammars for the following languages (using the notation from class):
   a. The empty language
   b. $\{0^i 1^j 2^k \mid i = j \vee j = k\}$
   c. $\{w \in \{0,1\}^* \mid w \textbf{ does not contain the substring } 000\}$
   d. $\{w \in \{a,b\}^* \mid w \textbf{ has twice as many } a\textbf{'s as } b\textbf{'s}\}$
   e. $\{a^n b^n a^n b^n \mid n \geq 0\}$

   (Note that not all of these are expressible with Ohm.)

4. Here's another look at the grammar for floating-point numerals (using single-letter variables for compactness):

   $n \longrightarrow d^+ \ f? \ e?$
   $f \longrightarrow$ `"."` d^+
   $e \longrightarrow$ (`"E"`|`"e"`) (`"+"`|`"-"`)? d^+
   $d \longrightarrow$ `"0".."9"`

   Give the (V, Σ, R, S)-definition of this grammar. (Note this means you will have to desugar the rules with `|`, `?`, and `+`.)

5. Give Turing Machines that recognize the following languages. If any of the languages below are Type-3, you may (and are encouraged to) give a FA in lieu of a TM recognizer, if the FA is simpler.
   a. $\{w \in \{a,b\}* \mid w \textbf{ ends with } abb\}$
   b. $\{w \in \{a,b\}^* \mid \#_a(w) = \#_b(w)\}$ (same number of a's as b's)
   c. $\{w \in \{a,b\}* \mid w \textbf{ alternates } a\textbf{'s and } b\textbf{'s}\}$
   d. $\{a^n b^n a^n b^n \mid n \geq 0\}$

6. Give Turing Machines that compute the following functions, where the input and output are binary numerals.
   a. $\lambda n.\ 2n + 2$
   b. one's complement
   c. The function described in Python as `lambda n: str(n)[1:-1]`

7. For the JavaScript/Python expression `5 * 3 - 1 ** 3`,
   a. Show a 3AC machine program to evaluate this expression, leaving the result in r_0

b. Show a Stack machine program to evaluate this expression, leaving the result on the top of the stack.

8. Characterize each of the following languages as either (a) regular, (b) context-free but not regular, (c) recursive but not context-free, (d) recursively enumerable but not recursive, or (e) not even recursively enumerable.
   a. $\{a^i b^j c^k \mid i > j > k\}$
   b. $\{a^i b^j c^k \mid i > j \wedge k \leq i - j\}$
   c. $\{\langle M \rangle \cdot w \mid M \text{ accepts } w\}$
   d. $\{G \mid G \text{ is context-free} \wedge L(G) = \varnothing\}$
   e. $\{a, b\}^* \{b\}^+$
   f. $\{\langle M \rangle \mid M \text{ does not halt }\}$
   g. $\{w \mid w \text{ is a decimal numeral divisible by 7}\}$
   h. $\{www \mid w \text{ is a string over the Unicode alphabet}\}$

For Your Project

Continue your compiler project in the public GitHub repository you created in the first assignment. You will be expanding your repo to have the following:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json
├── docs
|   └── ...
├── examples
|   └── ...
├── src
|   ├── (yourlanguagename).js
|   ├── (yourlanguagename).ohm
|   ├── compiler.js
|   ├── core.js
|   ├── analyzer.js
|   ├── optimizer.js        -- updated as below
|   └── generator.js        -- flesh this out!
└── test
    ├── compiler.test.js
    ├── grammar.test.js
```

```
├── analyzer.test.js
└── generator.test.js    -- new!
```

In this assignment you will be completing the following tasks:

- Complete the **code generator** portion of your compiler. You may output either C, C--, Java (source or JVM), LLVM, JavaScript, WebAssembly, the assembly language in the textbook, or code for any VM or IR you like (even one you create yourself), provided you check with me first. Implement the generator in src/generator.js and the tests in test/generator.test.js.
- Update the file src/optimizer.js to basically be a big no-op. Rewrite it like so:

```
export default function optimize(node) {
  return node
}
```

- Keep improving your README.
- Tidy up any issues found in the grammar and analyzer portions of your compiler (or any other component).
- Do any upgrades as is necessary for your companion website on GitHub pages.

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (80 pts)
  - ✅ Exercise 1 (10 pts)
  - ✅ Exercise 2 (10 pts)
  - ✅ Exercise 3 (10 pts)
  - ✅ Exercise 4 (10 pts)
  - ✅ Exercise 5 (10 pts)
  - ✅ Exercise 6 (10 pts)
  - ✅ Exercise 7 (10 pts)
  - ✅ Exercise 8 (10 pts)
- Compiler Project (20 pts)
  - ✅ All tests pass (2 pts)
  - ✅ Analyzer is complete (1 pt)
  - ✅ Generator is well structured (2 pts)

- ✅ Generator is complete (5 pts)
- ✅ You have “interesting” code generation (2 pts)
- ✅ You have a large number of tests for the code generator (2 pts)
- ✅ The generated code is readable (1 pt)
- ✅ Examples of generated code is on the README or companion site (2 pt)
- ✅ Test coverage is 100% (2 pts)
- ✅ Can run analyzer and generator from command line (1 pt)

Make sure to make all corrections on your project and project companion site from the previous assignments.